

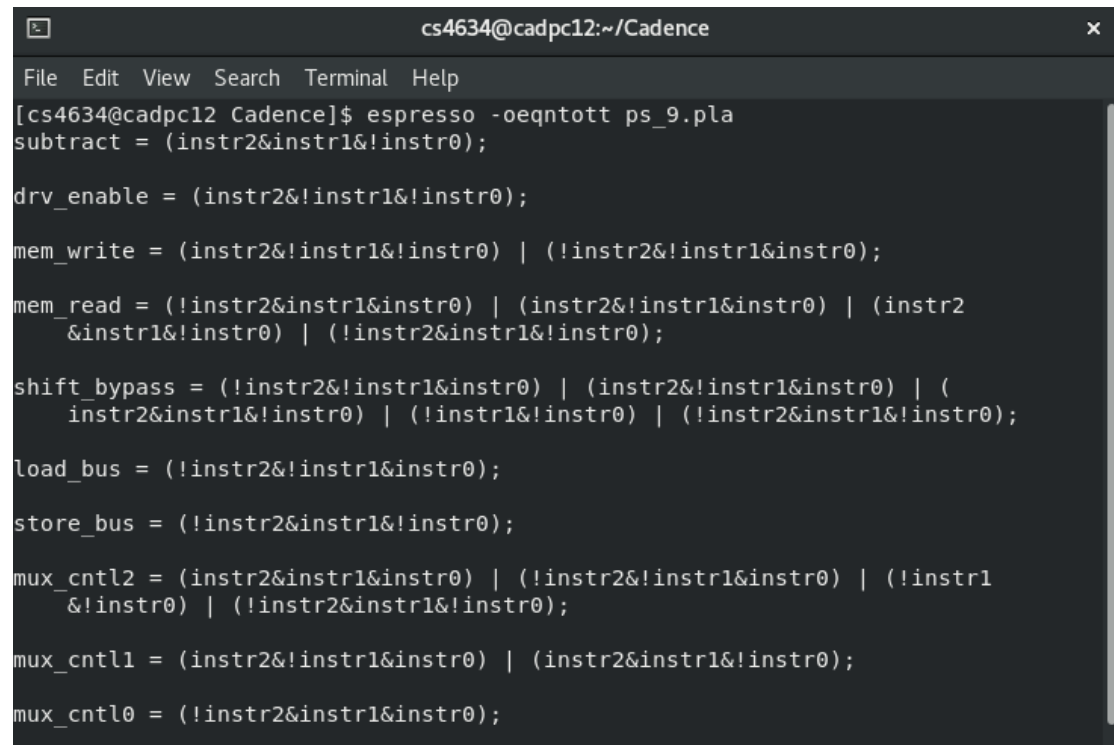
Microcontroller Write-Up

Jiaxuan Mao jm6111

Chenxi Shen cs4634

Design Hierarchy

PLA

A screenshot of a terminal window with a dark background. The title bar at the top reads 'cs4634@cadpc12:~/Cadence' and has a close button on the right. Below the title bar is a menu bar with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The main area of the terminal shows a series of logic equations for a PLA, starting with a command prompt '[cs4634@cadpc12 Cadence]\$ espresso -oeqntott ps_9.pla'. The equations are: 'subtract = (instr2&instr1&!instr0);', 'drv_enable = (instr2&!instr1&!instr0);', 'mem_write = (instr2&!instr1&!instr0) | (!instr2&instr1&instr0);', 'mem_read = (!instr2&instr1&instr0) | (instr2&!instr1&instr0) | (instr2 &instr1&!instr0) | (!instr2&instr1&!instr0);', 'shift_bypass = (!instr2&!instr1&instr0) | (instr2&!instr1&instr0) | (instr2&instr1&!instr0) | (!instr1&!instr0) | (!instr2&instr1&!instr0);', 'load_bus = (!instr2&!instr1&instr0);', 'store_bus = (!instr2&instr1&!instr0);', 'mux_cntl2 = (instr2&instr1&instr0) | (!instr2&!instr1&instr0) | (!instr1 &!instr0) | (!instr2&instr1&!instr0);', 'mux_cntl1 = (instr2&!instr1&instr0) | (instr2&instr1&!instr0);', and 'mux_cntl0 = (!instr2&instr1&instr0);'.

```
cs4634@cadpc12:~/Cadence
File Edit View Search Terminal Help
[cs4634@cadpc12 Cadence]$ espresso -oeqntott ps_9.pla
subtract = (instr2&instr1&!instr0);

drv_enable = (instr2&!instr1&!instr0);

mem_write = (instr2&!instr1&!instr0) | (!instr2&instr1&instr0);

mem_read = (!instr2&instr1&instr0) | (instr2&!instr1&instr0) | (instr2
&instr1&!instr0) | (!instr2&instr1&!instr0);

shift_bypass = (!instr2&!instr1&instr0) | (instr2&!instr1&instr0) | (
instr2&instr1&!instr0) | (!instr1&!instr0) | (!instr2&instr1&!instr0);

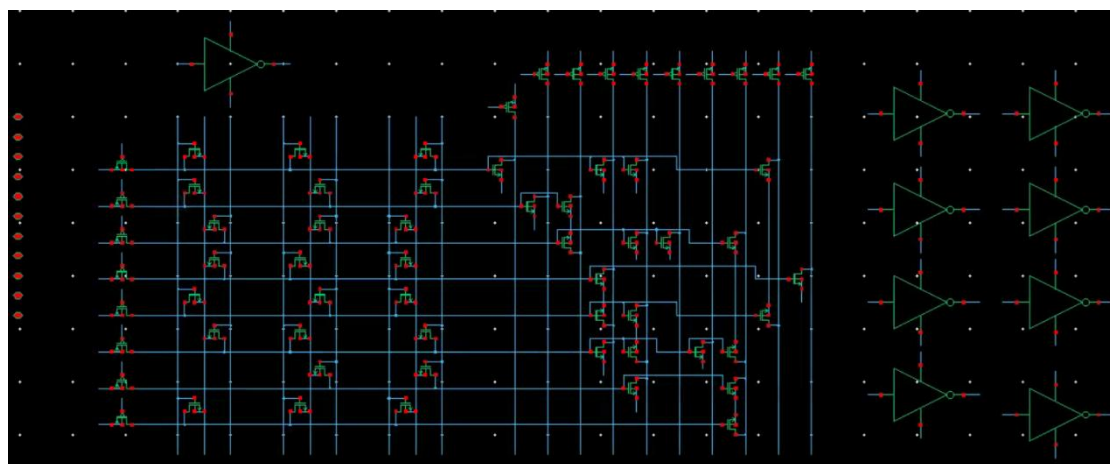
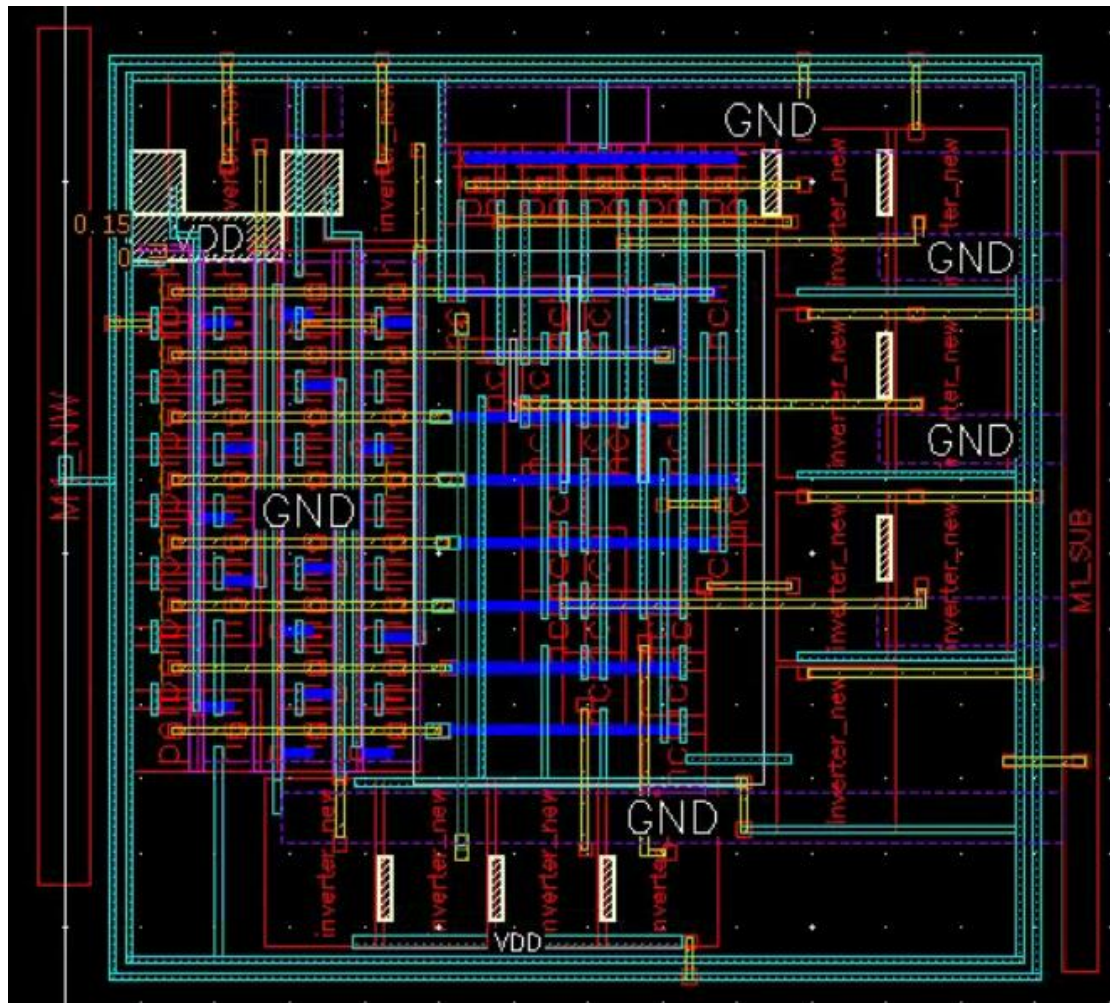
load_bus = (!instr2&!instr1&instr0);

store_bus = (!instr2&instr1&!instr0);

mux_cntl2 = (instr2&instr1&instr0) | (!instr2&!instr1&instr0) | (!instr1
&!instr0) | (!instr2&instr1&!instr0);

mux_cntl1 = (instr2&!instr1&instr0) | (instr2&instr1&!instr0);

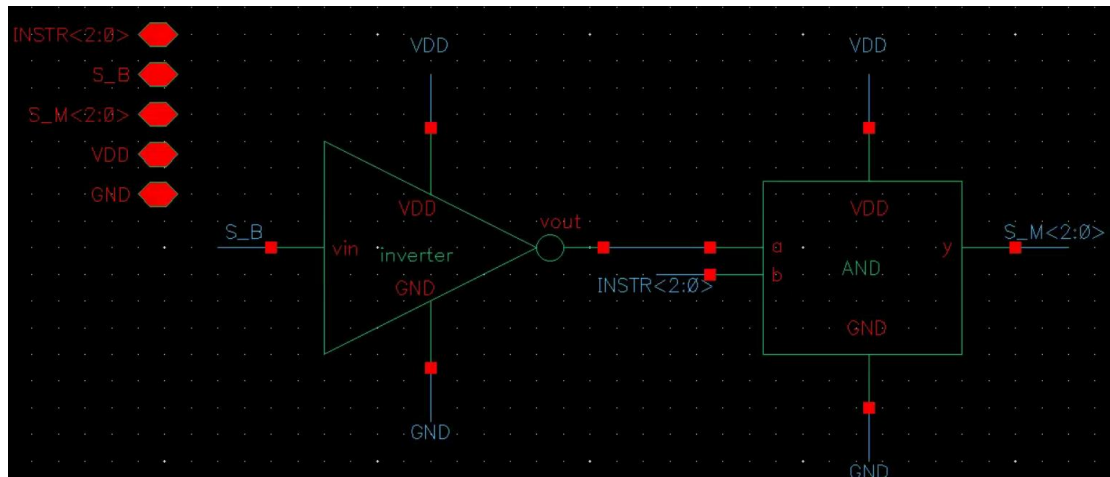
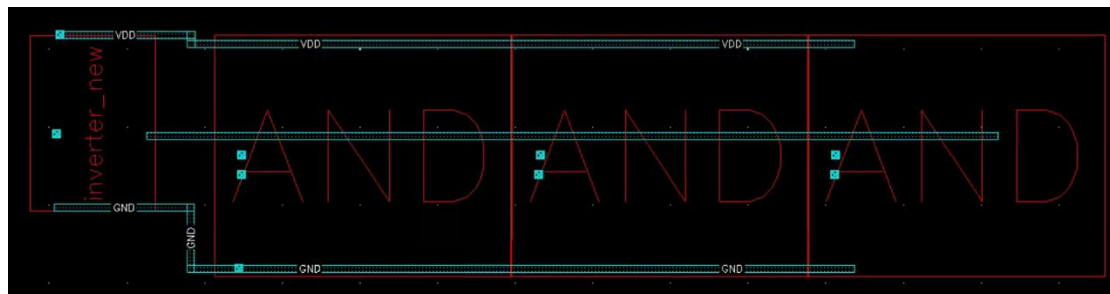
mux_cntl0 = (!instr2&instr1&instr0);
```



According to the instruction table we used the espresso to get the equations of different signals. The circuit features a structured AND-OR architecture optimized for logical regularity

We set 0.4um/0.4um for both pmos and nmos to get a strong ability of pulling down.

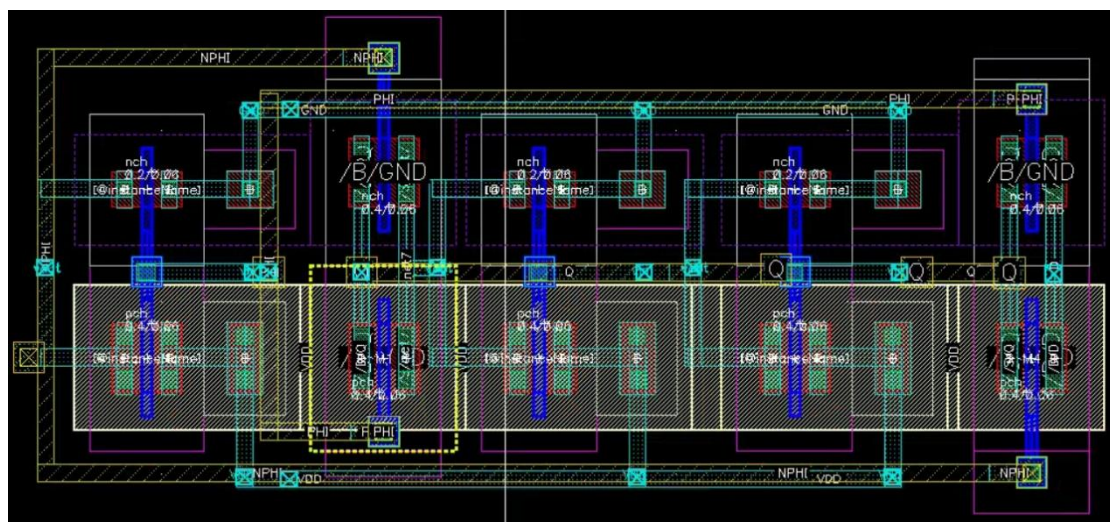
Shift amount

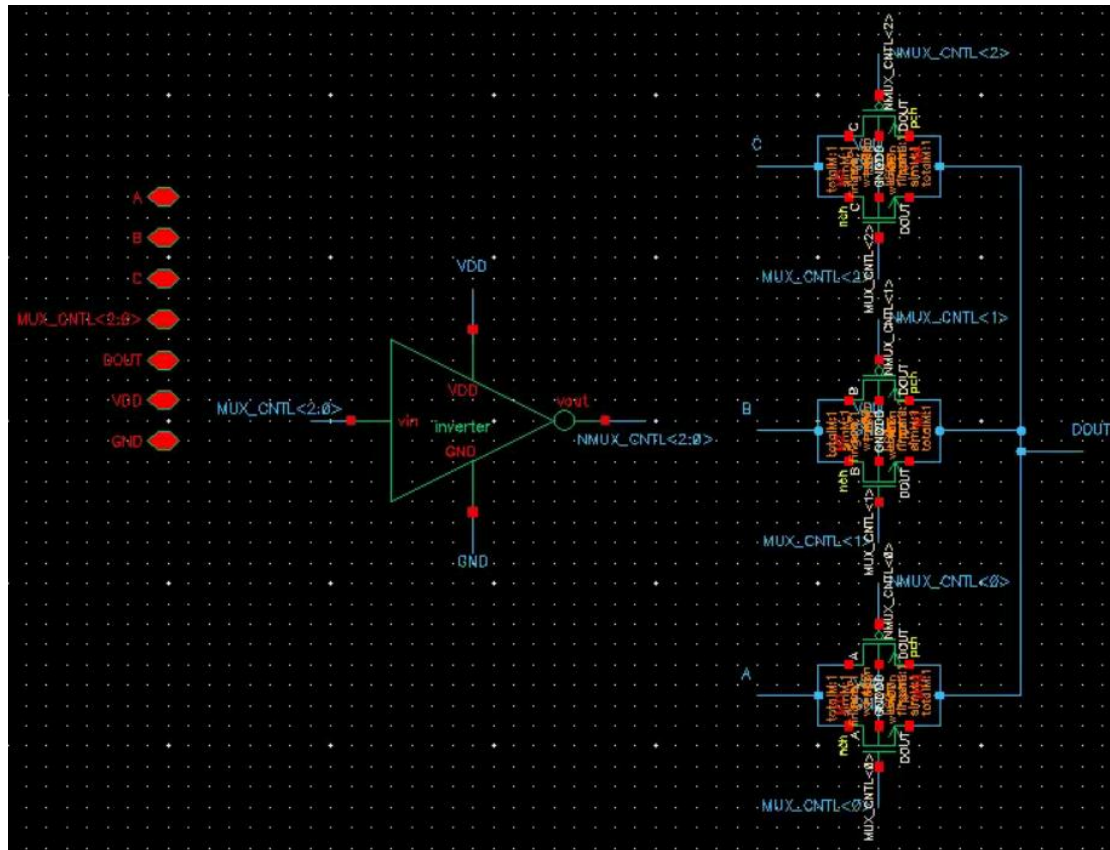


we use an and-gate to control the function of shifter. As when shifter_bypass = 1, the shifter cannot work no matter the value of the three signals representing the amount of shift.

We set 0.4um/0.2um for pmos and nmos as we did in our inverter to get a roughly smaller delay.

Latch



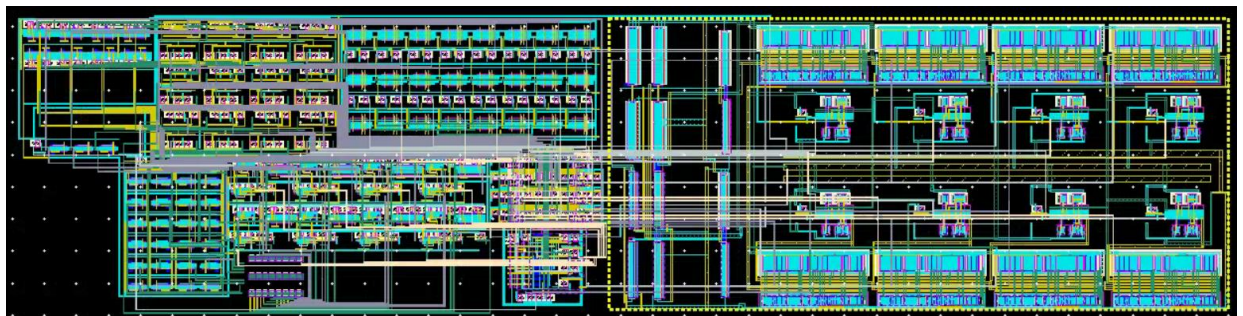


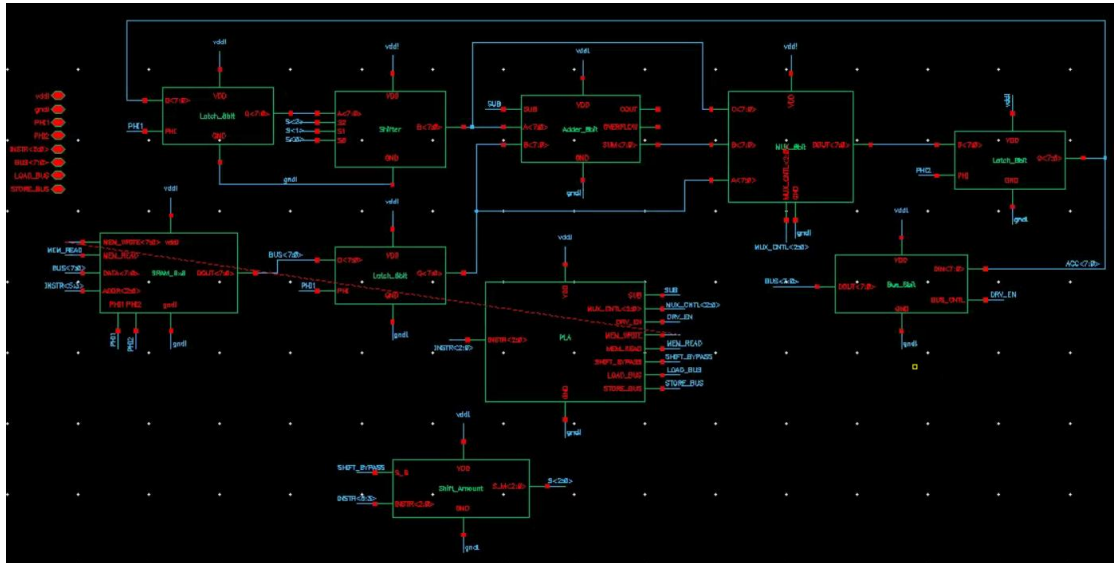
Bus driver

We used the read_cell of our last work in SRAM_8x8 to achieve the function of bus driver. The MUX_CNTL<3:0> is a one-hot signal to choose three pathes.

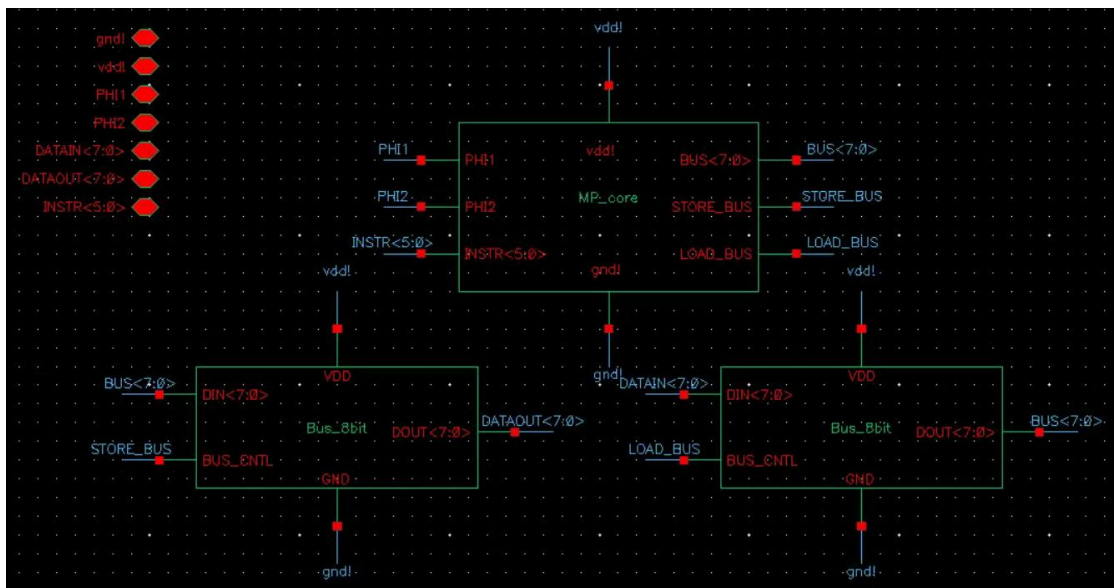
We set 0.4um/0.4um for pmos and nmos for the three TGs.

Microprocessor_core





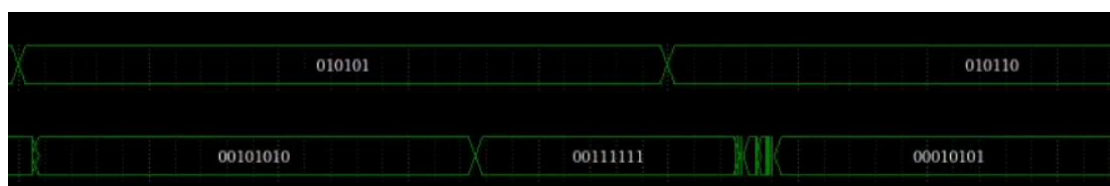
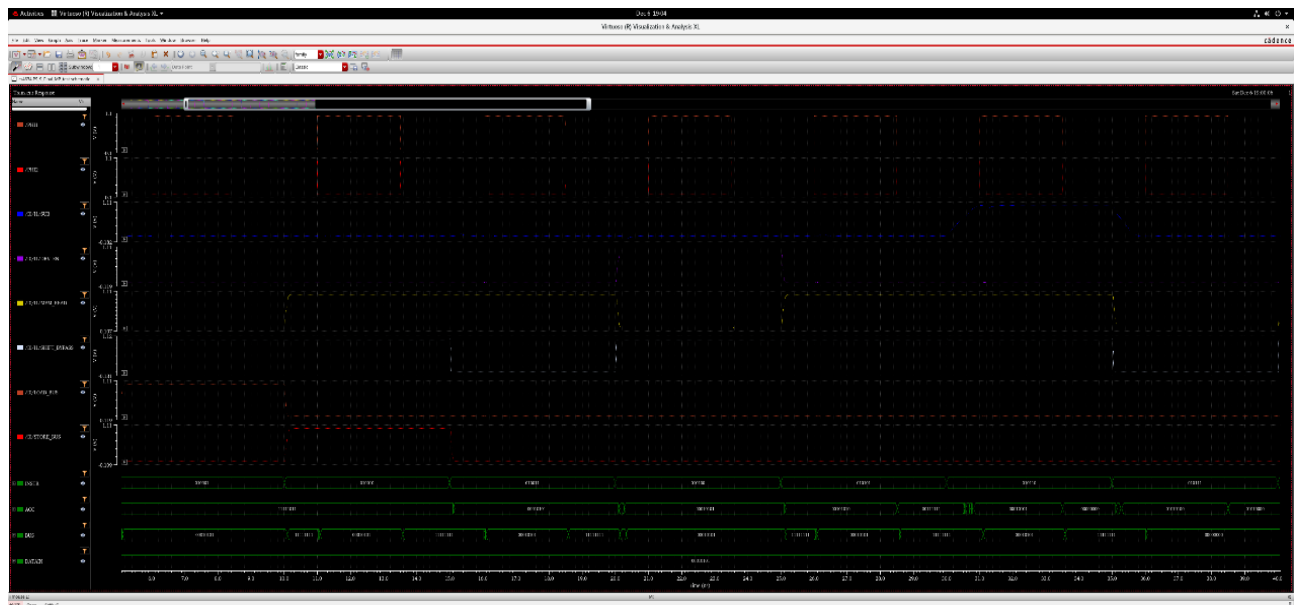
Top-level





The top-level schematic of our microcontroller is composed of the microprocessor_core and two external bus to input and output data.

Function



To simply test our microcontroller we set the INSTR<5:3> (the address) and the data fixed at

DRC Verified:

